

[INICIAL] TCArg 2026 Contest #1

A. Princesses and Princes

2 seconds, 256 megabytes

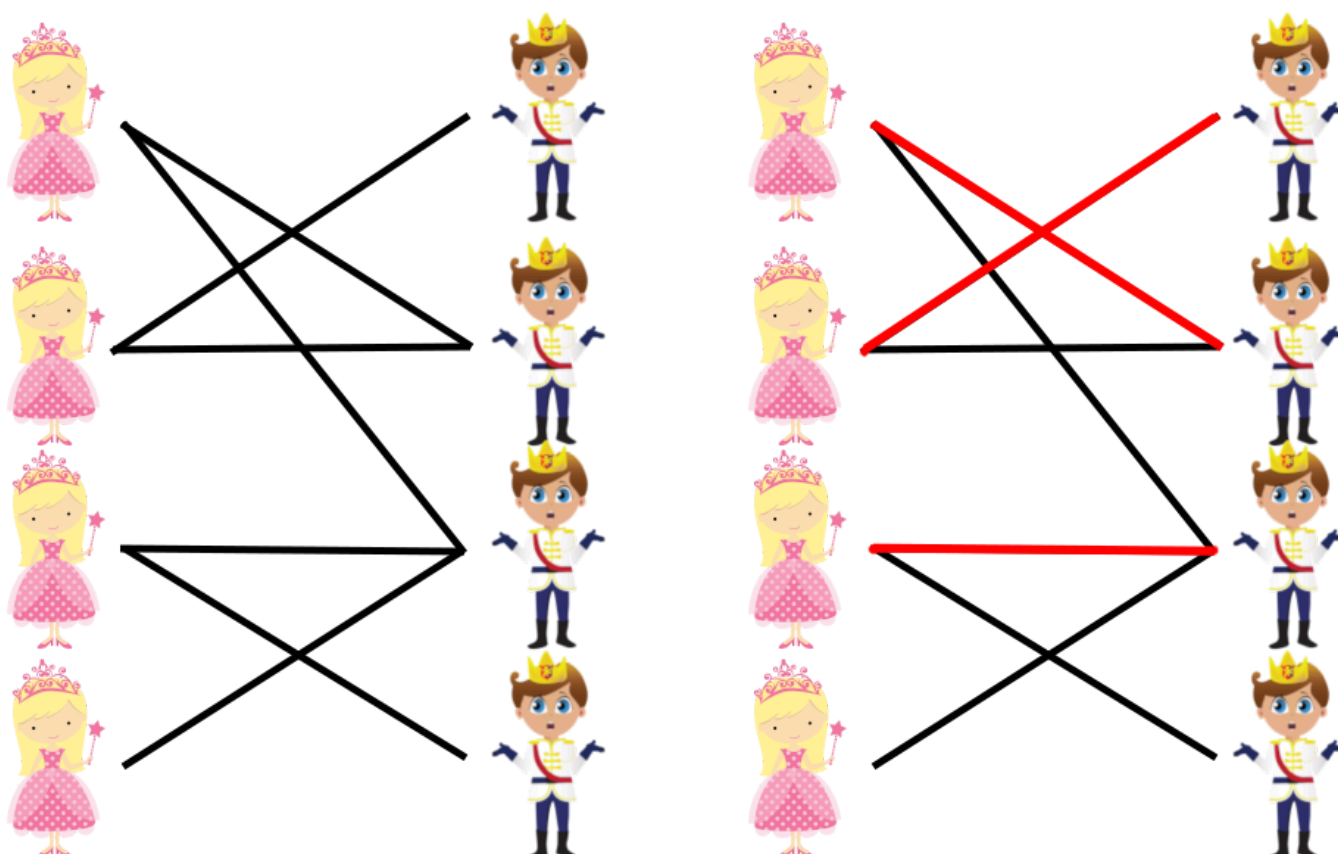
The King of Berland Polycarp LXXXIV has n daughters. To establish his power to the neighbouring kingdoms he wants to marry his daughters to the princes of these kingdoms. As a lucky coincidence there are n other kingdoms as well.

So Polycarp LXXXIV has enumerated his daughters from 1 to n and the kingdoms from 1 to n . For each daughter he has compiled a list of kingdoms princes of which she wanted to marry.

Polycarp LXXXIV is very busy, so he finds a couple for his daughters greedily one after another.

For the first daughter he takes **the kingdom with the lowest number from her list** and marries the daughter to their prince. For the second daughter he takes **the kingdom with the lowest number from her list, prince of which hasn't been taken already**. If there are no free princes in the list then the daughter marries nobody and Polycarp LXXXIV proceeds to the next daughter. The process ends after the n -th daughter.

For example, let there be 4 daughters and kingdoms, the lists daughters have are [2, 3], [1, 2], [3, 4], [3], respectively.



In that case daughter 1 marries the prince of kingdom 2, daughter 2 marries the prince of kingdom 1, daughter 3 marries the prince of kingdom 3, leaving daughter 4 nobody to marry to.

Actually, before starting the marriage process Polycarp LXXXIV has the time to convince one of his daughters that some prince is also worth marrying to. Effectively, that means that he can add exactly one kingdom to exactly one of his daughter's list. **Note that this kingdom should not be present in the daughter's list.**

Polycarp LXXXIV wants to increase the number of married couples.

Unfortunately, what he doesn't have the time for is determining what entry to add. If there is no way to increase the total number of married couples then output that the marriages are already optimal. Otherwise, find such an entry that the total number of married couples increases if Polycarp LXXXIV adds it.

If there are multiple ways to add an entry so that the total number of married couples increases then print any of them.

For your and our convenience you are asked to answer t independent test cases.

Input

The first line contains a single integer t ($1 \leq t \leq 10^5$) — the number of test cases.

Then t test cases follow.

The first line of each test case contains a single integer n ($1 \leq n \leq 10^5$) — the number of daughters and the number of kingdoms.

Each of the next n lines contains the description of each daughter's list. The first integer k ($0 \leq k \leq n$) is the number of entries in the i -th daughter's list. After that k distinct integers follow $g_i[1], g_i[2], \dots, g_i[k]$ ($1 \leq g_i[j] \leq n$) — the indices of the kingdoms in the list **in the increasing order** ($g_i[1] < g_i[2] < \dots < g_i[k]$).

It's guaranteed that the total number of daughters over all test cases does not exceed 10^5 .

It's also guaranteed that the total number of kingdoms in lists over all test cases does not exceed 10^5 .

Output

For each test case print the answer to it.

Print "IMPROVE" in the first line if Polycarp LXXXIV can add some kingdom to some of his daughter's list so that the total number of married couples increases. The second line then should contain two integers — the index of the daughter and the index of the kingdom Polycarp LXXXIV should add to that daughter's list.

If there are multiple ways to add an entry so that the total number of married couples increases then print any of them.

Otherwise the only line should contain one word "OPTIMAL".

input
5 4 2 2 3 2 1 2 2 3 4 1 3 2 0 0 3 3 1 2 3 3 1 2 3 3 1 2 3 1 1 1 4 1 1 1 2 1 3 1 4
output
IMPROVE 4 4 IMPROVE 1 1 OPTIMAL OPTIMAL OPTIMAL

The first test case is depicted in the statement. Adding the fourth kingdom to the list of the fourth daughter makes her marry the prince of the fourth kingdom.

In the second test case any new entry will increase the number of marriages from 0 to 1.

In the third and the fourth test cases there is no way to add an entry.

In the fifth test case there is no way to change the marriages by adding any entry.

B. Cubes for Masha

1 second, 256 megabytes

Absent-minded Masha got set of n cubes for her birthday.

At each of 6 faces of each cube, there is exactly one digit from 0 to 9. Masha became interested what is the largest natural x such she can make using her new cubes all integers from 1 to x .

To make a number Masha can rotate her cubes and put them in a row. After that, she looks at upper faces of cubes from left to right and reads the number.

The number can't contain leading zeros. It's not required to use all cubes to build a number.

Pay attention: Masha can't make digit 6 from digit 9 and vice-versa using cube rotations.

Input

In first line integer n is given ($1 \leq n \leq 3$) — the number of cubes, Masha got for her birthday.

Each of next n lines contains 6 integers a_{ij} ($0 \leq a_{ij} \leq 9$) — number on j -th face of i -th cube.

Output

Print single integer — maximum number x such Masha can make any integers from 1 to x using her cubes or 0 if Masha can't make even 1.

input
3 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7
output
87

input
3 0 1 3 5 6 8 1 2 4 5 7 8 2 3 4 6 7 9
output
98

In the first test case, Masha can build all numbers from 1 to 87, but she can't make 88 because there are no two cubes with digit 8.

C. Diverse Team

1 second, 256 megabytes

There are n students in a school class, the rating of the i -th student on Codeforces is a_i . You have to form a team consisting of k students ($1 \leq k \leq n$) such that the ratings of all team members **are distinct**.

If it is impossible to form a suitable team, print "NO" (without quotes). Otherwise print "YES", and then print k distinct numbers which should be the indices of students in the team you form. If there are multiple answers, print any of them.

Input

The first line contains two integers n and k ($1 \leq k \leq n \leq 100$) — the number of students and the size of the team you have to form.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$), where a_i is the rating of i -th student.

Output

If it is impossible to form a suitable team, print "NO" (without quotes). Otherwise print "YES", and then print k distinct integers from 1 to n which should be the indices of students in the team you form. All the ratings of the students in the team should be distinct. You may print the indices in any order. If there are multiple answers, print any of them.

Assume that the students are numbered from 1 to n .

input
5 3 15 13 15 15 12
output
YES 1 2 5

input
5 4 15 13 15 15 12
output
NO

input
4 4 20 10 40 30
output
YES 1 2 3 4

All possible answers for the first example:

- {1 2 5}
- {2 3 5}
- {2 4 5}

Note that the order does not matter.

D. Sasha and Magnetic Machines

1 second, 256 megabytes

One day Sasha visited the farmer 2D and his famous magnetic farm. On this farm, the crop grows due to the influence of a special magnetic field. Maintaining of the magnetic field is provided by n machines, and the power of the i -th machine is a_i .

This year 2D decided to cultivate a new culture, but what exactly he didn't say. For the successful growth of the new culture, it is necessary to slightly change the powers of the machines. 2D can **at most once** choose an arbitrary integer x , then choose one machine and reduce the power of its machine by x times, and at the same time increase the power of one another machine by x times (powers of all the machines must stay **positive integers**). Note that he may not do that if he wants. More formally, 2D can choose two such indices i and j , and one integer x such that x is a divisor of a_i , and change powers as following: $a_i = \frac{a_i}{x}$, $a_j = a_j \cdot x$

Sasha is very curious, that's why he wants to calculate the **minimum** total power the farmer can reach. There are too many machines, and Sasha can't cope with computations, help him!

Input

The first line contains one integer n ($2 \leq n \leq 5 \cdot 10^4$) — the number of machines.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$) — the powers of the machines.

Output

Print one integer — minimum total power.

input
5 1 2 3 4 5
output
14

input
4 4 2 4 4
output
14

input
5 2 4 2 3 7
output
18

In the first example, the farmer can reduce the power of the 4-th machine by 2 times, and increase the power of the 1-st machine by 2 times, then the powers will be: [2, 2, 3, 2, 5].

In the second example, the farmer can reduce the power of the 3-rd machine by 2 times, and increase the power of the 2-nd machine by 2 times. At the same time, the farmer can leave is be as it is and the total power won't change.

In the third example, it is optimal to leave it be as it is.

E. New Year and Counting Cards

1 second, 256 megabytes

Your friend has n cards.

You know that each card has a lowercase English letter on one side and a digit on the other.

Currently, your friend has laid out the cards on a table so only one side of each card is visible.

You would like to know if the following statement is true for cards that your friend owns: "If a card has a vowel on one side, then it has an even digit on the other side." More specifically, a vowel is one of 'a', 'e', 'i', 'o' or 'u', and even digit is one of '0', '2', '4', '6' or '8'.

For example, if a card has 'a' on one side, and '6' on the other side, then this statement is true for it. Also, the statement is true, for example, for a card with 'b' and '4', and for a card with 'b' and '3' (since the letter is not a vowel). The statement is false, for example, for card with 'e' and '5'. You are interested if the statement is true for all cards. In particular, if no card has a vowel, the statement is true.

To determine this, you can flip over some cards to reveal the other side. You would like to know what is the minimum number of cards you need to flip in the worst case in order to verify that the statement is true.

Input

The first and only line of input will contain a string s ($1 \leq |s| \leq 50$), denoting the sides of the cards that you can see on the table currently. Each character of s is either a lowercase English letter or a digit.

Output

Print a single integer, the minimum number of cards you must turn over to verify your claim.

input
ee
output
2

input
z
output
0

input
0ay1
output
2

In the first sample, we must turn over both cards. Note that even though both cards have the same letter, they could possibly have different numbers on the other side.

In the second sample, we don't need to turn over any cards. The statement is vacuously true, since you know your friend has no cards with a vowel on them.

In the third sample, we need to flip the second and fourth cards.

F. National Project

2 seconds, 256 megabytes

Your company was appointed to lay new asphalt on the highway of length n . You know that every day you can either repair one unit of the highway (lay new asphalt over one unit of the highway) or skip repairing.

Skipping the repair is necessary because of the climate. The climate in your region is periodical: there are g days when the weather is good and if you lay new asphalt these days it becomes high-quality pavement; after that, the weather during the next b days is bad, and if you lay new asphalt these days it becomes low-quality pavement; again g good days, b bad days and so on.

You can be sure that you start repairing at the start of a good season, in other words, days $1, 2, \dots, g$ are good.

You don't really care about the quality of the highway, you just want to make sure that **at least half of the highway** will have high-quality pavement. For example, if the $n = 5$ then at least 3 units of the highway should have high quality; if $n = 4$ then at least 2 units should have high quality.

What is the minimum number of days is needed to finish the repair of **the whole highway**?

Input

The first line contains a single integer T ($1 \leq T \leq 10^4$) — the number of test cases.

Next T lines contain test cases — one per line. Each line contains three integers n, g and b ($1 \leq n, g, b \leq 10^9$) — the length of the highway and the number of good and bad days respectively.

Output

Print T integers — one per test case. For each test case, print the minimum number of days required to repair **the whole highway if at least half of it should have high quality**.

input
3 5 1 1 8 10 10 1000000 1 1000000
output
5 8 499999500000

In the first test case, you can just lay new asphalt each day, since days $1, 3, 5$ are good.

In the second test case, you can also lay new asphalt each day, since days $1-8$ are good.

G. Candies and Two Sisters

1 second, 256 megabytes

There are two sisters Alice and Betty. You have n candies. You want to distribute these n candies between two sisters in such a way that:

- Alice will get a ($a > 0$) candies;
- Betty will get b ($b > 0$) candies;

- each sister will get some **integer** number of candies;
- Alice will get a greater amount of candies than Betty (i.e. $a > b$);
- all the candies will be given to one of two sisters (i.e. $a + b = n$).

Your task is to calculate the number of ways to distribute exactly n candies between sisters in a way described above. Candies are indistinguishable.

Formally, find the number of ways to represent n as the sum of $n = a + b$, where a and b are positive integers and $a > b$.

You have to answer t independent test cases.

Input

The first line of the input contains one integer t ($1 \leq t \leq 10^4$) — the number of test cases. Then t test cases follow.

The only line of a test case contains one integer n ($1 \leq n \leq 2 \cdot 10^9$) — the number of candies you have.

Output

For each test case, print the answer — the number of ways to distribute exactly n candies between two sisters in a way described in the problem statement. If there is no way to satisfy all the conditions, print 0.

input
6 7 1 2 3 2000000000 763243547
output
3 0 0 1 999999999 381621773

For the test case of the example, the 3 possible ways to distribute candies are:

- $a = 6, b = 1$;
- $a = 5, b = 2$;
- $a = 4, b = 3$.

H. Phoenix and Beauty

2 seconds, 256 megabytes

Phoenix loves beautiful arrays. An array is beautiful if all its subarrays of length k have the same sum. A subarray of an array is any sequence of consecutive elements.

Phoenix currently has an array a of length n . He wants to insert some number of integers, possibly zero, into his array such that it becomes beautiful. The inserted integers must be between 1 and n inclusive. Integers may be inserted anywhere (even before the first or after the last element), and he is **not trying** to minimize the number of inserted integers.

Input

The input consists of multiple test cases. The first line contains an integer t ($1 \leq t \leq 50$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq k \leq n \leq 100$).

The second line of each test case contains n space-separated integers ($1 \leq a_i \leq n$) — the array that Phoenix currently has. This array may or may not be already beautiful.

Output

For each test case, if it is impossible to create a beautiful array, print -1. Otherwise, print two lines.

The first line should contain the length of the beautiful array m ($n \leq m \leq 10^4$). You don't need to minimize m .

The second line should contain m space-separated integers ($1 \leq b_i \leq n$) — a beautiful array that Phoenix can obtain after inserting some, possibly zero, integers into his array a . You may print integers that weren't originally in array a .

If there are multiple solutions, print any. It's guaranteed that if we can make array a beautiful, we can always make it with resulting length no more than 10^4 .

input
<pre> 4 4 2 1 2 2 1 4 3 1 2 2 1 3 2 1 2 3 4 4 4 3 4 2 </pre>
output
<pre> 5 1 2 1 2 1 4 1 2 2 1 -1 7 4 3 2 1 4 3 2 </pre>

In the first test case, we can make array a beautiful by inserting the integer 1 at index 3 (in between the two existing 2s). Now, all subarrays of length $k = 2$ have the same sum 3. There exists many other possible solutions, for example:

- 2, 1, 2, 1, 2, 1
- 1, 2, 1, 2, 1, 2

In the second test case, the array is already beautiful: all subarrays of length $k = 3$ have the same sum 5.

In the third test case, it can be shown that we cannot insert numbers to make array a beautiful.

In the fourth test case, the array b shown is beautiful and all subarrays of length $k = 4$ have the same sum 10. There exist other solutions also.

I. Nastya and Rice

1 second, 256 megabytes

Nastya just made a huge mistake and dropped a whole package of rice on the floor. Mom will come soon. If she sees this, then Nastya will be punished.

In total, Nastya dropped n grains. Nastya read that each grain weighs some integer number of grams from $a - b$ to $a + b$, inclusive (numbers a and b are known), and the whole package of n grains weighs from $c - d$ to $c + d$ grams, inclusive (numbers c and d are known). The weight of the package is the sum of the weights of all n grains in it.

Help Nastya understand if this information can be correct. In other words, check whether each grain can have such a mass that the i -th grain weighs some integer number x_i ($a - b \leq x_i \leq a + b$), and in total they weigh from $c - d$ to $c + d$, inclusive ($c - d \leq \sum_{i=1}^n x_i \leq c + d$).

Input

The input consists of multiple test cases. The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

The next t lines contain descriptions of the test cases, each line contains 5 integers: n ($1 \leq n \leq 1000$) — the number of grains that Nastya counted and a, b, c, d ($0 \leq b < a \leq 1000, 0 \leq d < c \leq 1000$) — numbers that determine the possible weight of one grain of rice (from $a - b$ to $a + b$) and the possible total weight of the package (from $c - d$ to $c + d$).

Output

For each test case given in the input print "Yes", if the information about the weights is not inconsistent, and print "No" if n grains with masses from $a - b$ to $a + b$ cannot make a package with a total mass from $c - d$ to $c + d$.

input
<pre> 5 7 20 3 101 18 11 11 10 234 2 8 9 7 250 122 19 41 21 321 10 3 10 8 6 1 </pre>
output
<pre> Yes No Yes No Yes </pre>

In the first test case of the example, we can assume that each grain weighs 17 grams, and a pack 119 grams, then really Nastya could collect the whole pack.

In the third test case of the example, we can assume that each grain weighs 16 grams, and a pack 128 grams, then really Nastya could collect the whole pack.

In the fifth test case of the example, we can be assumed that 3 grains of rice weigh 2, 2, and 3 grams, and a pack is 7 grams, then really Nastya could collect the whole pack.

J. Union of Doubly Linked Lists

2 seconds, 256 megabytes

Doubly linked list is one of the fundamental data structures. A doubly linked list is a sequence of elements, each containing information about the previous and the next elements of the list. In this problem all lists have linear structure. I.e. each element except the first has exactly one previous element, each element except the last has exactly one next element. The list is not closed in a cycle.

In this problem you are given n memory cells forming one or more doubly linked lists. Each cell contains information about element from some list. Memory cells are numbered from 1 to n .

For each cell i you are given two values:

- l_i — cell containing previous element for the element in the cell i ;
- r_i — cell containing next element for the element in the cell i .

If cell i contains information about the element which has no previous element then $l_i = 0$. Similarly, if cell i contains information about the element which has no next element then $r_i = 0$.



Three lists are shown on the picture.

For example, for the picture above the values of l and r are the following: $l_1 = 4, r_1 = 7; l_2 = 5, r_2 = 0; l_3 = 0, r_3 = 0; l_4 = 6, r_4 = 1; l_5 = 0, r_5 = 2; l_6 = 0, r_6 = 4; l_7 = 1, r_7 = 0$.

Your task is to unite all given lists in a single list, joining them to each other in any order. In particular, if the input data already contains a single list, then there is no need to perform any actions. Print the resulting list in the form of values l_i, r_i .

Any other action, other than joining the beginning of one list to the end of another, can not be performed.

Input

The first line contains a single integer n ($1 \leq n \leq 100$) — the number of memory cells where the doubly linked lists are located.

Each of the following n lines contains two integers l_i, r_i ($0 \leq l_i, r_i \leq n$) — the cells of the previous and the next element of list for cell i . Value $l_i = 0$ if element in cell i has no previous element in its list. Value $r_i = 0$ if element in cell i has no next element in its list.

It is guaranteed that the input contains the correct description of a single or more doubly linked lists. All lists have linear structure: each element of list except the first has exactly one previous element; each element of list except the last has exactly one next element. Each memory cell contains information about one element from some list, each element of each list written in one of n given cells.

Output

Print n lines, the i -th line must contain two integers l_i and r_i — the cells of the previous and the next element of list for cell i after all lists from the input are united in a single list. If there are many solutions print any of them.

input
7 4 7 5 0 0 0 6 1 0 2 0 4 1 0
output
4 7 5 6 0 5 6 1 3 2 2 4 1 0

K. Construct the String

2 seconds, 256 megabytes

You are given three positive integers n, a and b . You have to construct a string s of length n consisting of lowercase Latin letters such that **each substring** of length a has **exactly** b distinct letters. It is guaranteed that the answer exists.

You have to answer t independent test cases.

Recall that the substring $s[l \dots r]$ is the string $s_l, s_{l+1}, \dots, s_r$ and its length is $r - l + 1$. In this problem you are only interested in substrings of length a .

Input

The first line of the input contains one integer t ($1 \leq t \leq 2000$) — the number of test cases. Then t test cases follow.

The only line of a test case contains three space-separated integers n , a and b ($1 \leq a \leq n \leq 2000, 1 \leq b \leq \min(26, a)$), where n is the length of the required string, a is the length of a substring and b is the required number of distinct letters in each substring of length a .

It is guaranteed that the sum of n over all test cases does not exceed 2000 ($\sum n \leq 2000$).

Output

For each test case, print the answer — such a string s of length n consisting of lowercase Latin letters that **each substring** of length a has **exactly** b distinct letters. If there are multiple valid answers, print any of them. It is guaranteed that the answer exists.

input
4 7 5 3 6 1 1 6 6 1 5 2 2
output
tleeelte qwerty vvvvvv abcde

In the first test case of the example, consider all the substrings of length 5:

- "tleeℓ": it contains 3 distinct (unique) letters,
- "leeℓt": it contains 3 distinct (unique) letters,
- "eeℓte": it contains 3 distinct (unique) letters.

L. Zero Quantity Maximization

2 seconds, 256 megabytes

You are given two arrays a and b , each contains n integers.

You want to create a new array c as follows: choose some real (i.e. not necessarily integer) number d , and then for every $i \in [1, n]$ let $c_i := d \cdot a_i + b_i$.

Your goal is to maximize the number of zeroes in array c . What is the largest possible answer, if you choose d optimally?

Input

The first line contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of elements in both arrays.

The second line contains n integers $a_1, a_2, ..., a_n$ ($-10^9 \leq a_i \leq 10^9$).

The third line contains n integers $b_1, b_2, ..., b_n$ ($-10^9 \leq b_i \leq 10^9$).

Output

Print one integer — the maximum number of zeroes in array c , if you choose d optimally.

input
5 1 2 3 4 5 2 4 7 11 3
output
2

input
3 13 37 39 1 2 3
output
2

input
4 0 0 0 0 1 2 3 4
output
0

input
3 1 2 -1 -6 -12 6
output
3

In the first example, we may choose $d = -2$.

In the second example, we may choose $d = -\frac{1}{13}$.

In the third example, we cannot obtain any zero in array c , no matter which d we choose.

In the fourth example, we may choose $d = 6$.

M. Powered Addition

1 second, 256 megabytes

You have an array a of length n . For every positive integer x you are going to perform the following operation during the x -th second:

- Select some distinct indices $i_1, i_2, \dots, i_k$ which are between 1 and n inclusive, and add 2^{x-1} to each corresponding position of a . Formally, $a_{i_j} := a_{i_j} + 2^{x-1}$ for $j = 1, 2, \dots, k$. **Note that you are allowed to not select any indices at all.**

You have to make a nondecreasing as fast as possible. Find the smallest number T such that you can make the array nondecreasing after at most T seconds.

Array a is nondecreasing if and only if $a_1 \leq a_2 \leq \dots \leq a_n$.

You have to answer t independent test cases.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains single integer n ($1 \leq n \leq 10^5$) — the length of array a . It is guaranteed that the sum of values of n over all test cases in the input does not exceed 10^5 .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$).

Output

For each test case, print the minimum number of seconds in which you can make a nondecreasing.

input
3 4 1 7 6 5 5 1 2 3 4 5 2 0 -4
output
2 0 3

In the first test case, if you select indices 3, 4 at the 1-st second and 4 at the 2-nd second, then a will become $[1, 7, 7, 8]$. There are some other possible ways to make a nondecreasing in 2 seconds, but you can't do it faster.

In the second test case, a is already nondecreasing, so answer is 0.

In the third test case, if you do nothing at first 2 seconds and select index 2 at the 3-rd second, a will become $[0, 0]$.

N. The Eternal Immortality

1 second, 256 megabytes

Even if the world is full of counterfeits, I still regard it as wonderful.

Pile up herbs and incense, and arise again from the flames and ashes of its predecessor — as is known to many, the phoenix does it like this.

The phoenix has a rather long lifespan, and reincarnates itself once every $a!$ years. Here $a!$ denotes the factorial of integer a , that is, $a! = 1 \times 2 \times \dots \times a$. Specifically, $0! = 1$.

Koyomi doesn't care much about this, but before he gets into another mess with oddities, he is interested in the number of times the phoenix will reincarnate in a timespan of $b!$ years, that is, $\frac{b!}{a!}$. Note that when $b \geq a$ this value is always integer.

As the answer can be quite large, it would be enough for Koyomi just to know **the last digit of the answer in decimal representation**. And you're here to provide Koyomi with this knowledge.

Input

The first and only line of input contains two space-separated integers a and b ($0 \leq a \leq b \leq 10^{18}$).

Output

Output one line containing a single decimal digit — the last digit of the value that interests Koyomi.

input
2 4
output
2

input
0 10
output
0

input
107 109
output
2

In the first example, the last digit of $\frac{4!}{2!} = 12$ is 2;

In the second example, the last digit of $\frac{10!}{0!} = 3628800$ is 0;

In the third example, the last digit of $\frac{109!}{107!} = 11772$ is 2.